Modelling Muon Transport in Iron with Neural ODEs and SDEs

Luis Felipe Cattelan
Physik-Institut, University of Zürich
Course project — Data Driven Modelling of Dynamical Systems

June 8, 2026

1 Introduction

When a charged particle crosses a block of matter it does not travel in a straight line at constant energy: it continuously loses momentum to the atomic electrons and is randomly deflected by the nuclei. Detector simulations such as Geant4 reproduce this in great detail, collision by collision, but at a high computational cost. The idea of this project is to ask whether a small neural network, trained on Geant4 output, can act as a fast *surrogate* for one elementary process: the change in a muon's momentum after it travels a short step through iron. Practically, such a learned per-step map could replace the expensive Monte-Carlo stepping inside a simulation; more importantly, it is a clean test for a conceptual question about deterministic versus stochastic modelling.

The model I want to learn is best stated by looking at the physics. A muon of momentum p that travels an arc length "step" through iron undergoes two things:

- it **loses momentum along its direction of travel** ($dP_z < 0$) by ionizing the atomic electrons
- it is **deflected sideways** (dP_t) by many small-angle Coulomb scatters off the nuclei

The object to be learned is therefore the map $(p_0, \text{step}) \mapsto p_1$, the momentum after one step. The catch is that the data are stochastic – every muon scatters differently – while a deterministic network outputs only one value per input. Can such a model still learn the process "in the mean", and where does it break down?

More formally, the muon's momentum along its path obeys an equation of motion of the form

$$\frac{dp}{ds} = \underbrace{q\,\hat{p} \times B}_{\text{Lorentz force (known)}} + \underbrace{f_\theta(p, \text{step})}_{\text{material transport (to learn)}}, \tag{1}$$

where $\hat{p} = p/|p|$: the first term is the deflection by a magnetic field B and the second is the change caused by the material (energy loss and scattering). The Lorentz term is known analytically and trivial to evaluate once the field is given, so the genuinely unknown, hard part is the material transport. The goal of this project is precisely to learn that transport term, f_θ, from the Geant4 data.

2 The model

2.1 Overview

I build two models that share the same neural backbone but differ in their output and training objective: a *deterministic* Neural ODE that learns the mean transport rate dp/ds, and a *stochastic*

Neural SDE that learns a drift and a diffusion. Both are small multilayer perceptrons (three hidden layers, 64 units, tanh) implemented in PyTorch. Before describing them I recall the reference physics they are compared against.

2.2 Reference physics (the formulae we compare to)

Mean energy loss – Bethe–Bloch [1].

$$-\left\langle \frac{dE}{dx} \right\rangle = K z^2 \frac{Z}{A} \frac{1}{\beta^2} \left[\tfrac{1}{2} \ln \frac{2 m_e c^2 \beta^2 \gamma^2 W_{\max}}{I^2} - \beta^2 - \frac{\delta(\beta\gamma)}{2} \right], \tag{2}$$

with $K = 0.307\,\mathrm{MeV\,cm^2/mol}$ and, for iron, $Z/A = 0.466$, $\rho = 7.87\,\mathrm{g/cm^3}$, $I = 286\,\mathrm{eV}$; $W_{\max}$ is the maximum single-collision energy transfer and δ the Sternheimer density-effect correction. For a relativistic muon $dE \approx c\,dP_z$, so $\langle |dP_z| \rangle \propto$ step.

Multiple scattering – Highland [2].

$$\theta_0 = \frac{13.6\,\mathrm{MeV}}{\beta c\, p}\, z \sqrt{x/X_0}\, \left[1 + 0.038 \ln\!\big(x z^2/(X_0 \beta^2) \big) \right], \tag{3}$$

with $X_0 = 1.76\,\mathrm{cm}$ for iron. The transverse momentum kick is $p_T \approx p\,\theta$, so $\mathrm{RMS}\,|p_T| = \sqrt{2}\,p\,\theta_0 \propto \sqrt{x}$.

Energy-loss fluctuations – Landau/Vavilov [3]. The energy loss in a thin layer is Landau-distributed (a sharp peak with a long tail from rare hard δ-rays). Its variance (Bohr) is

$$\sigma_E^2 = \tfrac{K}{2} \frac{Z}{A} \frac{z^2}{\beta^2} \rho x\, W_{\max} \big(1 - \tfrac{\beta^2}{2} \big). \tag{4}$$

2.3 Data and simulation setup

The data were produced with Geant4 [4], which transports muons through iron and stores, step by step along each trajectory, the momentum before and after the step; every row of the resulting table is one such single step. I work in a deliberately reduced setting: a single material (iron), no magnetic field (a field-free slab, so the only momentum changes come from the material) and no tracking of the spatial position – only the momenta. Of the physics I keep ionization energy loss (dP_z) and multiple Coulomb scattering (dP_t); radiative losses (bremsstrahlung etc.) are $\lesssim$ a few percent at these energies and are not modelled separately.

I take the muon to start along z, so the initial momentum is $p_0 = [0, 0, p_0]$ with $p_0 \in [10, 20]\,\mathrm{GeV}$, and after one step $p_1 = [dP_t,\, 0,\, p_0 + dP_z]$: the longitudinal component shrinks by the energy loss and a transverse kick of magnitude dP_t appears. The kick is placed along x because its azimuth is physically uniform and carries no information; the random direction is restored at inference by an azimuthal rotation. Concretely we simulate millions of muons: for each one a step size is drawn, and Geant4 returns that (random) step together with the resulting dP_z and dP_t.

For training we feed the network the logarithm of the step size and normalize all inputs, which is necessary for numerical stability.

2.4 Deterministic Neural ODE

The motion over arc length s obeys an equation of the form $dp/ds = f(p, s)$. Instead of hand-coding f from Bethe–Bloch and scattering theory, a Neural ODE [5] *learns* it: the rate is parametrized by the network,

$$\frac{dp}{ds} = f_\theta(p, \text{step}), \tag{5}$$

and the momentum after a finite step is obtained by a fourth-order Runge–Kutta (RK4) integration,

$$\begin{aligned} k_1 &= f_\theta(p_0,\, ds), \qquad k_2 = f_\theta\big(p_0 + \tfrac{ds}{2}k_1,\, ds\big),\\ k_3 &= f_\theta\big(p_0 + \tfrac{ds}{2}k_2,\, ds\big), \qquad k_4 = f_\theta\big(p_0 + ds\,k_3,\, ds\big),\\ \hat{p}_1 &= p_0 + \tfrac{ds}{6}\big(k_1 + 2k_2 + 2k_3 + k_4\big). \end{aligned} \tag{6}$$

Training is supervised on the single-step transitions (p_0, step, p_1): for each sample we integrate one RK4 step and minimize the mean squared error of the *relative* momentum change,

$$\mathcal{L}_{\text{MSE}} = \Big\langle \big\| \,(\hat{p}_1 - p_0)/|p_0| - (p_1 - p_0)/|p_0|\,\big\|^2 \Big\rangle. \tag{7}$$

Dividing by $|p_0|$ makes the target dimensionless and of order 10^{-3} independent of the momentum. The key property is that the minimizer of an MSE loss is the *conditional mean*: $f_\theta \to \mathbb{E}[dp \mid p_0, \text{step}]$. The deterministic model therefore learns the mean energy loss and the mean transverse kick, but discards all fluctuation around them. At inference a trajectory is generated by chaining RK4 steps, with an optional random azimuthal rotation after each step to mimic the random scattering direction.

2.5 Stochastic Neural SDE

I model the same transport as an Itô stochastic differential equation,

$$dp = \mu_\theta(p, s)\, ds + \sigma_\theta(p, s)\, dW, \qquad dW \sim \mathcal{N}(0, ds\,\mathbb{I}), \tag{8}$$

where the network outputs a drift μ and a (log-)diffusion σ. A single Euler–Maruyama step reads $\Delta p = \mu\, ds + \sigma\sqrt{ds}\,\xi$ with $\xi \sim \mathcal{N}(0, \mathbb{I})$. The one-step transition is the diagonal Gaussian $p_1 \mid p_0 \sim \mathcal{N}(p_0 + \mu\, ds,\ \text{diag}(\sigma^2 ds))$, and I train by minimizing its negative log-likelihood (NLL). Unlike MSE, the NLL fits the mean *and* the spread. Because scattering is isotropic but the data fix the kick along one axis, we apply a random azimuthal rotation to each (p_0, p_1) pair during training; this forces the transverse drift to vanish and makes $\sigma_x = \sigma_y$, so the transverse spread is carried entirely by the diffusion term.

Why an SDE is the natural object. An ODE integrated by RK4 produces increments $\Delta p \propto ds$. Scattering requires $\Delta p_t \propto \sqrt{ds}$, i.e. a rate $dp_t/ds \propto 1/\sqrt{ds}$ that diverges as $ds \to 0$ – impossible for a bounded network. The SDE puts the $\sqrt{ds}$ in the noise term, so the network only learns a smooth σ.

2.6 Training, tools and sources

Both models are trained on all 2×10^6 steps (a 90/10 train/validation split) with Adam (learning rate 10^{-3}), batches of 1024, gradient-norm clipping at 1, and a learning-rate-on-plateau schedule, for 10–15 epochs; I keep the lowest-validation checkpoint. I found the weight decay had to be set to

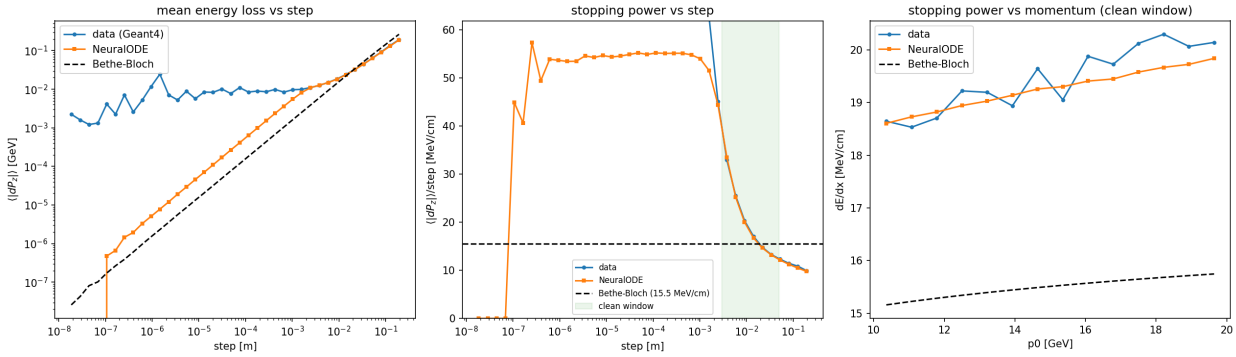

Figure 1: Energy loss vs. Bethe–Bloch. Left: mean $|dP_z|$ vs. step. Centre: stopping power vs. step (the small-step blow-up is the selection bias of Sec. 3.2). Right: stopping power vs. momentum in a step window. The NeuralODE (orange) tracks the data (blue) everywhere.

zero: at the usual 10^{-3} it is comparable to the signal itself (targets $\sim 10^{-3}$) and simply drives the predictions toward zero. The code is written in Python with PyTorch [7] and is available at `https://github.com/lfpc/ode_modelling`; the analytic reference curves use the PDG parametrizations [1, 2].

3 Analysis and validation

3.1 Mean energy loss vs. Bethe–Bloch

Figure 1 compares the model's predicted mean $|dP_z|$ with the data and Bethe–Bloch. The Neural ODE overlays the data across all step sizes. The naive stopping power averaged over a step window sits $\sim 25\%$ above the Bethe–Bloch value of 15.5 MeV/cm – but, as explained next, this is an artifact of the data, not of the model: restricting to large, self-averaging steps removes it. In the window 1–5 cm (Fig. 2) the data (15.47), model (15.25) and theory (15.48 MeV/cm) agree at the percent level and show the same relativistic rise with momentum.

3.2 The step-length selection bias

The stopping power is a *mean over many collisions*; a step recovers it only if it contains enough of them. For short steps the loss is Landau-distributed and dominated by whether a rare hard collision occurred. Moreover, Geant4 places step boundaries *at* interaction points, so short steps are preferentially those containing a hard δ-ray (biased high) while long steps are soft-collision-only stretches (biased low). Measuring the local $\langle|dP_z|\rangle$/step in narrow windows confirms this: it falls from ~ 500 MeV/cm at 0.1 mm, through the Bethe–Bloch value at 1–3 cm, down to ~ 9 MeV/cm at 10 cm. The sub-linear fit $dP_z \propto \text{step}^{0.62}$ is the fingerprint of this artifact, not of the physics. The model is faithful; the bias lives in the data, and it is removed by evaluating only where the data self-averages.

3.3 Scattering vs. Highland

The left panel of Fig. 3 compares the SDE's learned diffusion with the data RMS$|p_T|$ and Highland. In the self-averaging window the three agree closely: data 23.3, model 22.8, Highland 21.5 MeV. The $\sim 8\%$ excess of the data over Highland is expected: Highland describes only the Gaussian core,

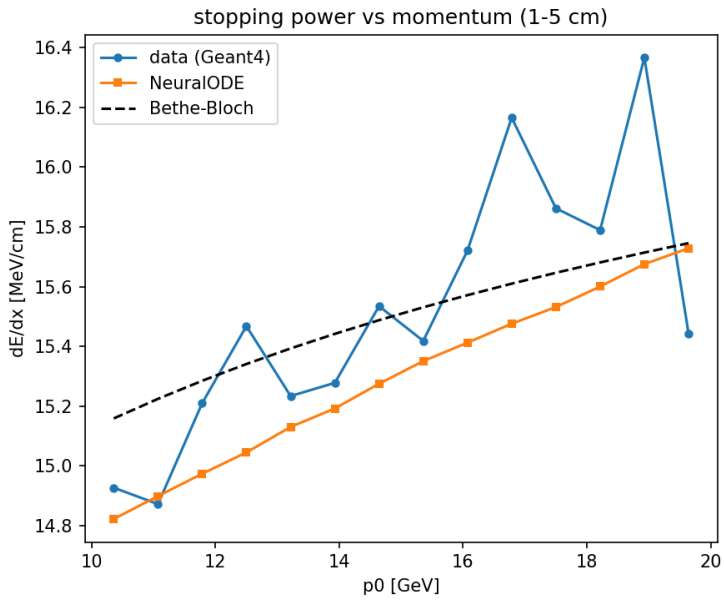

Figure 2: Stopping power vs. momentum, restricted to the self-averaging window (1–5 cm): data, model and Bethe–Bloch agree to $\sim 1\%$.

whereas the RMS is sensitive to the non-Gaussian large-angle (Rutherford) tail. Removing the top 2% of events (which carry 14.5% of the variance) brings the data core to 21.5 MeV, matching Highland exactly.

3.4 Energy-loss straggling vs. Vavilov

The right panel of Fig. 3 compares the SDE's $\sigma_z\sqrt{ds}$ with the data spread and Bohr/Vavilov, Eq. (4). The theory (70.5 MeV) is within $\sim 13\%$ of the data (81.3 MeV) with the correct $\sqrt{\text{step}}$ slope, but the Gaussian SDE underestimates it (53.4 MeV). This is the expected failure of a Gaussian to represent a heavy Landau tail: the likelihood-optimal Gaussian width is anchored to the bulk and cannot account for the rare catastrophic losses.

4 Discussion

This project started from a simple question – can a deterministic network learn a stochastic physical process "in the mean" – and ended up being a clean demonstration of the difference between drift and diffusion. The most satisfying result is that, after fixing two non-obvious issues (the input normalization, and restricting the comparison to self-averaging steps), the learned energy loss lands essentially on top of the data from Geant4.

The two main difficulties were as instructive as the successes. First, the *step-length selection bias* in the Geant4 data: I initially read the $\sim 25\%$ excess in the stopping power as a model error, and only understood it after measuring the local dE/dx as a function of step and realizing that short steps in Geant4 are statistically special. A cleaner dataset (fixed step length, or the continuous loss reported separately from δ-rays) would avoid this entirely. Second, the *Gaussian-versus-Landau* limitation: the SDE captures the variance of each channel but not the skew of the energy-loss distribution nor the power-law scattering tail. A natural next step would be a heavy-tailed noise

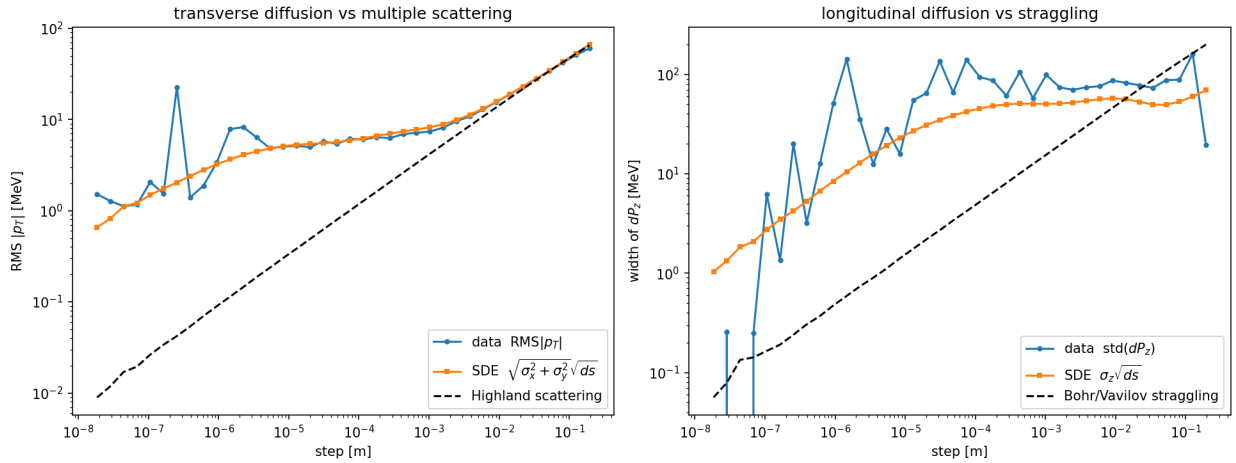

Figure 3: SDE diffusion vs. fluctuation theory. Left: transverse width vs. Highland (excellent agreement). Right: longitudinal width vs. Bohr/Vavilov; the Gaussian SDE captures the scale but underestimates the heavy-tailed width.

(for example modelling $\log dP_z$, or using a Student-t or normalizing-flow noise), which I expect would close the remaining gap in the straggling.

A more principled framework, and the one I would adopt if I continued this work, is the *Neural Jump Stochastic Differential Equation* of Jia and Benson [6]. There the state evolves by a continuous neural flow (a drift, and optionally a diffusion) but is also subject to discrete *jumps* that occur at random times drawn from a learned point process, each jump applying a learned (stochastic) increment to the state. This matches the physics almost exactly: the continuous flow is the soft, frequent collisions – the smooth dE/dx and the Gaussian core of the scattering that my models already capture – while the *jumps* are precisely the rare hard events I miss, namely the energetic δ-rays that build the Landau tail of the energy loss and the large-angle Rutherford scatters that build the non-Gaussian tail of the deflection. Splitting the dynamics into a smooth part and a jump part this way, rather than forcing a single Gaussian to absorb both, is the natural way to reproduce the heavy tails that are the one ingredient missing from the present model.

In summary, the mean energy loss is learned essentially exactly; the scattering width is captured well by the Gaussian SDE; and only the heavy tail of the energy-loss fluctuations is genuinely missed.

References

[1] Particle Data Group, R. L. Workman *et al.*, "Review of Particle Physics" , *Prog. Theor. Exp. Phys.* (2022).

[2] V. L. Highland, "Some practical remarks on multiple scattering," *Nucl. Instrum. Meth.* **129**, 497 (1975); G. R. Lynch and O. I. Dahl, *Nucl. Instrum. Meth.* **B58**, 6 (1991).

[3] L. D. Landau, "On the energy loss of fast particles by ionization," *J. Phys. USSR* **8**, 201 (1944).

[4] S. Agostinelli *et al.*, "Geant4 – a simulation toolkit," *Nucl. Instrum. Meth.* **A506**, 250 (2003).

[5] R. T. Q. Chen, Y. Rubanova, J. Bettencourt and D. Duvenaud, "Neural Ordinary Differential Equations," *NeurIPS* (2018).

[6] J. Jia and A. R. Benson, "Neural Jump Stochastic Differential Equations," *NeurIPS* (2019).

[7] A. Paszke *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," *NeurIPS* (2019).